

560. 和为 K 的子数组

1.问题

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium (44.77%)	2738	0	-	-	0

▼ Tags

数组 | 哈希表 | 前缀和

▼ Companies

给你一个整数数组 `nums` 和一个整数 `k`，请你统计并返回 该数组中和为 `k` 的子数组的个数。

子数组是数组中元素的连续非空序列。

示例 1:

```
输入：nums = [1,1,1], k = 2
输出：2
```

示例 2:

```
输入：nums = [1,2,3], k = 3
输出：2
```

提示:

- `1 <= nums.length <= 2 * 104`
- `1000 <= nums[i] <= 1000`
- `107 <= k <= 107`

2.解析

560. 和为 K 的子数组

解题思路

- 这道题可以使用**前缀和 + 哈希表**的方法高效解决：
- 维护一个变量 `sum` 记录从数组开始到当前位置的累加和（前缀和）
 - 使用哈希表记录每个前缀和出现的次数
 - 对于当前前缀和 `sum`，如果之前存在前缀和 `sum - k`，说明存在一段子数组的和为 `k`

代码实现

```
class Solution {
    public int subarraySum(int[] nums, int k) {
        // 记录结果：和为k的子数组数量
        int count = 0;
        // 记录前缀和
        int sum = 0;
        // 哈希表：key是前缀和，value是该前缀和出现的次数
```

```

HashMap<Integer, Integer> prefixSumCount = new HashMap<>();

// 初始化：前缀和为0的情况出现1次（空数组）
prefixSumCount.put(0, 1);

for (int num : nums) {
    // 累加前缀和
    sum += num;

    // 查找是否存在sum-k的前缀和
    // 如果存在，说明从之前某个位置到当前位置的子数组和为k
    if (prefixSumCount.containsKey(sum - k)) {
        count += prefixSumCount.get(sum - k);
    }

    // 更新前缀和的出现次数
    prefixSumCount.put(sum, prefixSumCount.getOrDefault(sum, 0) + 1);
}

return count;
}
}

```

示例分析

示例1: nums = [1,1,1], k = 2

1. 初始状态：

- count = 0
- sum = 0
- prefixSumCount = {0:1}

2. 遍历数组：

- i=0: num=1
 - sum = 1
 - 检查sum-k=1-2=-1是否在哈希表中 → 不存在
 - 更新哈希表：prefixSumCount = {0:1, 1:1}
- i=1: num=1
 - sum = 2
 - 检查sum-k=2-2=0是否在哈希表中 → 存在，count += 1
 - 更新哈希表：prefixSumCount = {0:1, 1:1, 2:1}
- i=2: num=1
 - sum = 3
 - 检查sum-k=3-2=1是否在哈希表中 → 存在，count += 1
 - 更新哈希表：prefixSumCount = {0:1, 1:1, 2:1, 3:1}

3. 最终结果：count = 2

- 对应子数组：[1,1] 和 [1,1,1]中的后两个元素

示例2: nums = [1,2,3], k = 3

1. 初始状态：

- count = 0

- $sum = 0$
- $prefixSumCount = \{0:1\}$

2. 遍历数组：

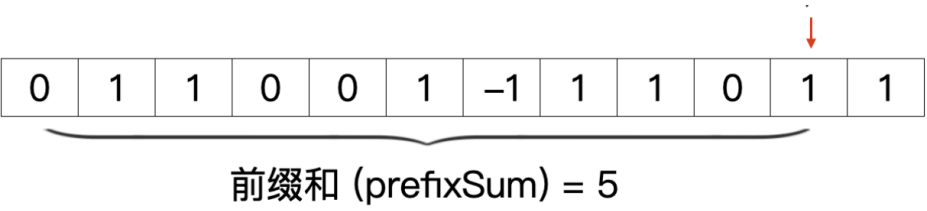
- $i=0: num=1$
 - $sum = 1$
 - 检查 $sum-k=1-3=-2$ 是否在哈希表中 → 不存在
 - 更新哈希表： $prefixSumCount = \{0:1, 1:1\}$
- $i=1: num=2$
 - $sum = 3$
 - 检查 $sum-k=3-3=0$ 是否在哈希表中 → 存在， $count += 1$
 - 更新哈希表： $prefixSumCount = \{0:1, 1:1, 3:1\}$
- $i=2: num=3$
 - $sum = 6$
 - 检查 $sum-k=6-3=3$ 是否在哈希表中 → 存在， $count += 1$
 - 更新哈希表： $prefixSumCount = \{0:1, 1:1, 3:1, 6:1\}$

3. 最终结果： $count = 2$

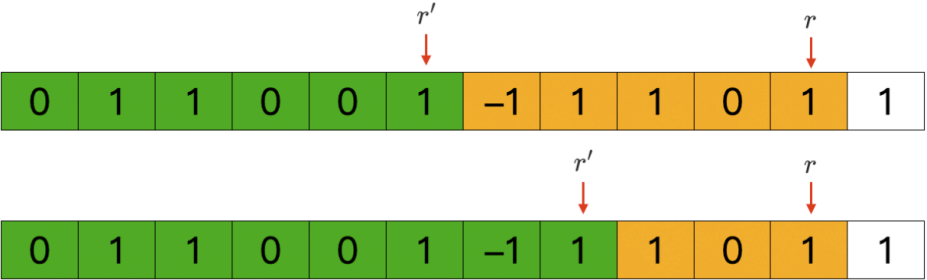
- 对应子数组： $[1,2]$ 和 $[3]$

复杂度分析

- **时间复杂度：** $O(n)$ ，其中 n 是数组长度，我们只需要遍历一次数组
- **空间复杂度：** $O(n)$ ，最坏情况下，哈希表需要存储 n 个不同的前缀和

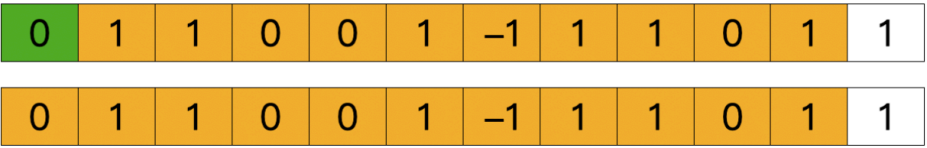


我们先找到右边界 $r' < r$ 且前缀和为 $3 (5 - k)$ 的两个绿色子数组如下所示，相应黄色子数组即为所求。



我们可以采用 hash map 记录这些前缀和的数量。但要注意：

- 如果 $k = 5$ ，即当前 r 处前缀和恰为 k ，按以上方式寻找的是前缀和为 $0 (k - k)$ 的数量。如下所示，两个黄色子数组都能满足要求，但 hash map 中前缀和为 0 的数量正常为 1 ，即少记录了一次绿色子数组不存在的情况。所以我们应该先在 hash map 中将其设为 1 ，而后可能增加。



- 因为 $r' < r$ ，所以我们应在 r 处累计过数量之后再将 r 处的前缀和计入 hash map，详见算法实现。

算法实现

```
Kotlin Java C++ Python
class Solution {
    fun subarraySum(nums: IntArray, k: Int): Int {
        var preSum = 0
        val freq = mutableMapOf<Int, Int>()
        freq[0] = 1
        var cnt = 0

        for(v in nums){
            preSum += v
            cnt += freq[preSum - k] ?: 0
            freq[preSum] = (freq[preSum] ?: 0) + 1
        }

        return cnt
    }
}
```

还可以简化如下。

```
Kotlin Java C++ Python
class Solution {
    fun subarraySum(nums: IntArray, k: Int): Int {
        var preSum = 0
        val freq = mutableMapOf<Int, Int>()
        var cnt = 0

        for(v in nums){
            for(v in nums){
                preSum += v
                cnt += freq[preSum - k] ?: 0
                freq[preSum] = (freq[preSum] ?: 0) + 1
            }
        }

        return cnt
    }
}
```

还可以简化如下。

```
Kotlin Java C++ Python
class Solution {
    fun subarraySum(nums: IntArray, k: Int): Int {
        var preSum = 0
        val freq = mutableMapOf<Int, Int>()
        var cnt = 0

        for(v in nums){
            // 先存入前一个位置的前缀和，恰好也预先计入了 freq[0] = 1
            freq[preSum] = (freq[preSum] ?: 0) + 1

            // 后更新当前位置前缀和，该前缀和此时未计入 freq，限制 r' < r
            preSum += v
            cnt += freq[preSum - k] ?: 0
        }

        return cnt
    }
}
```